Лабораторная работа
Параллельное решение СЛАУ алгоритмом Гаусса с использованием OpenMP

Студент группы ИА332 Рубцов Роман Кириллович

Новосибирск, 2026

1 Цель работы

Реализация последовательного и параллельного (OpenMP) алгоритмов решения систем линейных алгебраических уравнений (СЛАУ) методом Гаусса. Профилировка прямого и обратного хода алгоритма. Проведение экспериментального анализа масштабируемости параллельного алгоритма при различном числе потоков (1, 2, 4, 6, 8, 16, 32) и оценка точности вычислений.

2 Конфигурация стенда

- **Процессор:** Intel Core i5-10400F (6 ядер, 12 потоков).
- **Среда разработки:** GCC (флаги `-O3 -fopenmp`).
- **Размер матрицы:** $n = 1500$.
- **Тип данных:** `double` (двойная точность).

3 Описание параллельного алгоритма

Метод Гаусса состоит из двух этапов: прямого и обратного хода. Информационные зависимости алгоритма накладывают строгие ограничения на распараллеливание.

3.1 1. Прямой ход (исключение неизвестных)

На прямом ходе матрица приводится к верхнетреугольному виду. Вычислительная сложность данного этапа имеет порядок $O(n^3)$.

- Внешний цикл по ведущей строке i распараллеливать нельзя, так как каждая следующая итерация зависит от результатов предыдущей.

- Параллелизуется внутренний цикл по строкам k, в котором происходит вычитание ведущей строки.
- Так как количество обрабатываемых строк $(n - 1 - i)$ уменьшается на каждом шаге, для балансировки нагрузки применяется стратегия `schedule(guided)`, динамически уменьшающая размер выдаваемых потокам блоков строк.

3.2 2. Обратный ход (нахождение неизвестных)

На обратном ходе осуществляется подстановка найденных значений неизвестных. Вычислительная сложность этого этапа составляет $O(n^2)$.

- Нахождение текущей неизвестной x_i выполняется строго одним потоком внутри директивы `#pragma omp single` для избежания гонок данных (data race). В конце блока стоит неявный барьер синхронизации.
- После нахождения x_i, потоки параллельно (`schedule(static)`) вычитают её вклад из правых частей всех уравнений, расположенных выше строки i.

4 Результаты измерений

Матрица генерировалась с диагональным преобладанием, точное решение задавалось как вектор из единиц. Максимальная абсолютная погрешность (Max Error) для всех запусков составила 1.15×10^{-14}, что подтверждает теоретическую точность машинных вычислений прямого метода Гаусса. Время усреднялось по 5 пускам (среднегеометрическое).

Таблица 1: Профилировка времени выполнения (в секундах)

Потоки	**Общее время (Total)**	**Прямой ход (Forward)**	**Обратный ход (Backward)**
1	0.743518	0.742234	0.001283
2	0.499069	0.497699	0.001364
4	0.414687	0.413435	0.001250
6	0.365907	0.364412	0.001494
8	0.358301	0.356714	0.001586
16	0.472879	0.417953	0.054911
32	0.548939	0.445104	0.103735

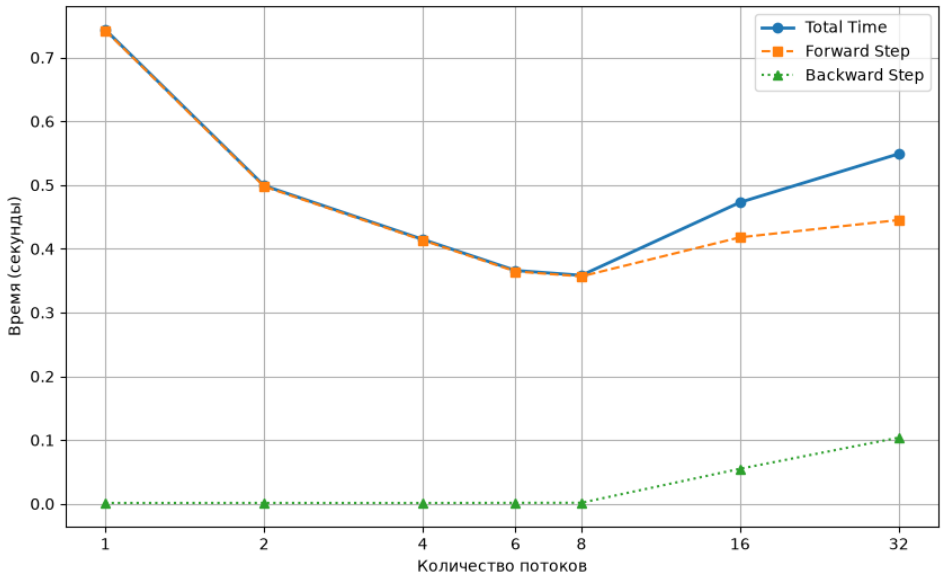


Рис. 1: Зависимость времени прямого и обратного хода от числа потоков

5 Анализ данных

5.1 Масштабируемость прямого хода

- На интервале от 1 до 6 потоков наблюдается стабильное ускорение вычислений прямого хода за счет распределения операций $O(n^3)$ по физическим ядрам процессора.
- Оптимум достигается на 8 потоках (0.356 с), где вычислительные ресурсы логических ядер (Hyper-Threading) утилизируются максимально эффективно.

5.2 Деградация при избытке потоков (over-subscription)

- На 16 и 32 потоках (больше аппаратного максимума стенда в 12 потоков) общее время выполнения возрастает до 0.548 с.
- Данная деградация наиболее ярко выражена в **обратном ходе**. Ввиду его низкой вычислительной сложности $O(n^2)$ на каждую операцию вычитания приходится высокая доля накладных расходов.
- Алгоритм требует двух барьеров синхронизации на каждой из n итераций обратного хода. При 16–32 потоках время, затрачиваемое рантаймом OpenMP на синхронизацию (busy-waiting) и переключение контекста ОС (context switch), значительно превышает время полезной работы, что вызывает рост времени обратного хода с 1.5 мс (8 потоков) до 103 мс (32 потока).

6 Вывод

Успешно реализован и профилирован метод Гаусса с использованием OpenMP. Подтверждена высокая точность метода. Показано, что для вычислительно плотных участков с распараллеливанием по данным (прямой ход) эффективно применение динамической балансировки `schedule(guided)`. Однако для участков с малым объемом вычислений и высокой частотой синхронизаций (обратный ход) избыточное число потоков приводит к резкому падению производительности из-за накладных расходов. Наилучшее время достигнуто на 8 потоках.